

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	4
ВВЕДЕНИЕ	6
1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ.....	9
1.1 Обзор зарубежных исследований	9
1.1.1 Исследования в области браузерной идентификации	9
1.1.2 Исследования систем фильтрации электронной почты	11
1.1.3 Исследования в области автоматизации и имитации поведения	12
1.2 Обзор отечественных исследований	14
1.3 Общая характеристика проблемы исследования	16
1.3.1 Формулировка проблемы	17
1.3.2 Ограничения существующих подходов.....	17
1.3.3 Обоснование выбранного подхода	18
1.3.4 Научная новизна и практическая значимость	19
2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ	21
2.1 Основные методы решения проблемы и их характеристики	21
2.1.1 Методы генерации цифровых отпечатков браузера.....	21
2.1.2 Методы имитации поведения пользователя	22
2.1.3 Методы обхода систем детекции автоматизации	23
2.1.4 Методы балансировки нагрузки и управления аккаунтами	24
2.2 Основные технологии решения проблемы и их характеристики	25
2.2.1 Фреймворки браузерной автоматизации	26
2.2.2 Технологии сокрытия автоматизации.....	27
2.2.3 Брокеры сообщений для распределённой обработки задач	27

2.2.4 Системы оркестрации процессов	28
2.3 Алгоритм выбора методов и технологий решения проблемы.....	29
2.3.1 Критерии выбора	29
2.3.2 Пошаговый алгоритм выбора	30
2.3.3 Обоснование выбранного технологического стека.....	31
3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ	33
3.1 Общая характеристика предлагаемого решения.....	33
3.2 Функциональные требования к предлагаемому решению	35
3.2.1 Модуль управления аккаунтами и безопасностью.....	36
3.2.2 Модуль рассылок и оркестрации	36
3.2.3 Модуль коммуникации	37
3.3 Нефункциональные требования к предлагаемому решению	38
3.3.1 Требования к производительности и масштабируемости	38
3.3.2 Требования к безопасности и анонимности.....	39
3.3.3 Требования к отказоустойчивости и надёжности	40
ЗАКЛЮЧЕНИЕ	42
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	44

ВВЕДЕНИЕ

Email-маркетинг остаётся одним из наиболее эффективных каналов цифрового маркетинга с показателем возврата инвестиций (ROI) до 4200% по данным отраслевых исследований. Однако в период 2022–2024 годов отрасль столкнулась с системной проблемой снижения доставляемости: средний показатель inbox placement снизился с 83% до 76%, что означает недоставку каждого четвёртого письма.

Причиной данной тенденции стало комплексное ужесточение политик крупнейших почтовых провайдеров. С февраля 2024 года Google ввёл обязательные требования для массовых отправителей: наличие протоколов аутентификации SPF и DKIM, настройка политики DMARC, показатель спам-жалоб менее 0,3%. Microsoft Outlook и Yahoo Mail внедрили аналогичные требования. Одновременно системы фильтрации на базе нейронных сетей и методов глубокого обучения достигли точности 99,9% в детекции нежелательных сообщений.

Исследования показывают, что только 3% доменов корректно внедрили комплекс протоколов аутентификации, что создаёт системное противоречие: провайдеры ужесточают требования, но большинство легитимных отправителей им не соответствует. Традиционные решения на базе протокола SMTP демонстрируют ограниченную эффективность в новых условиях, что определяет актуальность поиска альтернативных подходов.

Перспективным направлением является браузерная автоматизация с применением технологии имитации поведения пользователя (Human Behavior Simulation). Научные исследования подтверждают, что добавление поведенческих сигналов – траекторий движения мыши, паттернов скроллинга, динамики набора текста – позволяет обходить системы детекции автоматизации. Данный подход требует комплексного научного обоснования и формирования системы требований к программному обеспечению.

Степень разработанности проблемы. Теоретические основы исследования сформированы в работах зарубежных учёных: Vastel A., Laperdrix P. (browser fingerprinting), Amin Azad B., Nikiforakis N. (антибот-системы), Asien A., Fierrez J. (синтез траекторий мыши), Dada E. G. (спам-фильтрация). Среди отечественных исследователей следует отметить работы Яхнеевой И. В., Павловой А. В. (автоматизация маркетинга), Смирновой О. С. (детекция ботов), Машечкина И. В. (системы фильтрации). Вместе с тем комплексное применение методов имитации поведения к задаче автоматизации email-маркетинга остаётся недостаточно исследованным.

Объектом исследования являются процессы автоматизации email-маркетинга и технологии обеспечения доставляемости электронных сообщений.

Предметом исследования выступают методы и технологии имитации поведения пользователя для обхода систем автоматической детекции в контексте браузерной автоматизации.

Цель исследования – проведение комплексного анализа проблемы снижения доставляемости email-рассылок и формирование требований к программному комплексу автоматизации email-маркетинга с применением технологии имитации поведения пользователя.

Для достижения поставленной цели определены следующие задачи исследования:

- 1) провести анализ состояния исследований в области браузерной идентификации, систем фильтрации электронной почты и технологий имитации поведения пользователя;
- 2) выполнить сравнительный анализ методов и технологий решения проблемы доставляемости;
- 3) разработать алгоритм выбора технологического стека для системы браузерной автоматизации;
- 4) сформулировать функциональные и нефункциональные требования к программному комплексу.

Методы исследования: анализ научной литературы, систематизация и классификация, сравнительный анализ, моделирование, формализация требований.

Научная новизна исследования заключается в комплексном применении методов имитации поведения пользователя к задаче автоматизации email-маркетинга и разработке алгоритма выбора технологического стека, учитывающего специфику противодействия системам детекции.

Практическая значимость работы состоит в формировании теоретического фундамента и системы требований для разработки программного комплекса автоматизации email-маркетинга. Результаты исследования могут быть использованы при создании систем цифрового маркетинга, RPA-решений, инструментов браузерной автоматизации.

Структура работы. Отчёт состоит из введения, трёх глав, заключения и списка использованных источников. В первой главе проведён обзор состояния исследований в предметной области. Во второй главе представлен анализ технологий и методов решения проблемы. В третьей главе сформулированы требования к программному комплексу.

1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.1 Обзор зарубежных исследований

Проблематика автоматизации email-маркетинга с применением технологий имитации поведения пользователя находится на пересечении нескольких научных направлений: браузерной идентификации (browser fingerprinting), систем фильтрации электронной почты, робототехнической автоматизации процессов (RPA) и биометрического анализа поведения. Анализ зарубежных публикаций позволяет выявить актуальное состояние исследований и определить теоретическую базу для разрабатываемого решения.

1.1.1 Исследования в области браузерной идентификации

Технология браузерной идентификации (browser fingerprinting) представляет собой метод сбора информации о конфигурации браузера и устройства пользователя для формирования уникального идентификатора без использования cookies. Данная технология активно применяется антифрод-системами почтовых провайдеров для выявления автоматизированной активности.

Vastel et al. [1] в работе «FP-Scanner: The Privacy Implications of Browser Fingerprint Inconsistencies», опубликованной на конференции USENIX Security Symposium 2018, представили инструмент FP-SCANNER, предназначенный для обнаружения модификаций цифрового отпечатка браузера. Исследователи продемонстрировали возможность выявления оригинальных значений изменённых атрибутов через кросс-атрибутный анализ. Ключевым выводом работы является то, что несогласованность параметров fingerprint (например, несоответствие между заявленным User-Agent и реальными характеристиками

рендеринга) служит надёжным признаком попытки маскировки. Данный вывод критически важен для разработки устойчивых профилей браузера.

Развитием данного направления стала работа Vastel et al. [2] «FP-Crawlers: Studying the Resilience of Browser Fingerprinting to Block Crawlers» (NDSS MADWeb Workshop, 2020). Исследование 10 000 наиболее посещаемых сайтов (Alexa Top 10K) выявило, что 31,96% из них используют fingerprinting для блокировки автоматизированных обращений. Авторы идентифицировали конкретные техники детекции headless-браузеров: проверка флага navigator.webdriver, анализ списка плагинов (navigator.plugins), верификация поддержки WebGL. Важным результатом является то, что стандартный браузер Chrome без модификаций блокируется в 75% случаев на основе поведенческого и сетевого анализа, что подтверждает необходимость комплексного подхода к сокрытию автоматизации.

Amin Azad et al. [3] в исследовании «Web Runner 2049: Evaluating Third-Party Anti-bot Services» (DIMVA 2020, Springer) провели первую комплексную оценку коммерческих антибот-сервисов методами gray-box и black-box тестирования. Результаты показали, что более 75% защищённых сайтов используют fingerprinting в качестве основного механизма детекции. Принципиально важным для настоящего исследования является вывод авторов о том, что добавление поведенческих сигналов – движений мыши и паттернов скроллинга – позволяет обходить значительную часть систем детекции. Данный результат служит прямым подтверждением эффективности подхода Human Behavior Simulation.

В продолжение этого исследования Amin Azad et al. [4] опубликовали работу «Taming the Shape Shifter: Detecting Anti-fingerprinting Browsers» (DIMVA 2020), в которой представили категоризацию методов модификации fingerprint: JavaScript-инъекция, нативная подмена параметров (native spoofing), полное пересоздание браузерного окружения. Авторы выявили детектируемые следы каждого метода: для JavaScript-инъекции – нестандартное поведение toString()-функций в DOM, для нативной подмены –

несоответствия ANGLE-строк графического рендеринга. Результаты исследования необходимо учитывать при разработке системы генерации fingerprint-профилей.

1.1.2 Исследования систем фильтрации электронной почты

Понимание механизмов работы систем фильтрации электронной почты является необходимым условием для разработки решений, обеспечивающих высокую доставляемость сообщений.

Dada et al. [5] в систематическом обзоре «Machine learning for email spam filtering: review, approaches and open research problems» (Heliyon, 2019) провели комплексный анализ алгоритмов спам-фильтрации, применяемых крупнейшими почтовыми провайдерами – Gmail, Yahoo, Outlook. Исследователи установили, что система фильтрации Gmail достигает точности 99,9% благодаря применению нейросетевых моделей и методов машинного обучения. Фильтры анализируют сотни признаков, включая репутацию домена отправителя, структуру заголовков сообщения, историю взаимодействий получателя с отправителем. В качестве перспективного направления противодействия авторы рекомендуют применение методов deep adversarial learning.

Nightingale [6] в техническом отчёте NIST «Email Authentication Mechanisms: DMARC, SPF and DKIM» (NIST Technical Note 1945, 2017) представил авторитетное описание протоколов аутентификации электронной почты. SPF (Sender Policy Framework, RFC 7208) обеспечивает аутентификацию источника отправки через DNS-записи. DKIM (DomainKeys Identified Mail, RFC 6376) гарантирует целостность сообщения посредством криптографической подписи. DMARC (Domain-based Message Authentication, Reporting and Conformance, RFC 7489) объединяет оба механизма и обеспечивает обратную связь о результатах проверки. Данный документ

является обязательной базой для понимания технических причин не доставки email-сообщений.

Tatang et al. [7] в работе «The Evolution of DNS-based Email Authentication: Measuring Adoption and Finding Flaws» (RAID '21, ACM) провели крупномасштабное измерение внедрения протоколов аутентификации на выборке Top 1M доменов за период 2018–2020 гг. Результаты выявили критически низкий уровень корректного внедрения: SPF настроен у ~50% доменов, DMARC – у ~11%, DKIM – у ~13%. При этом ~70% политик DMARC используют режим «None», не обеспечивающий реальной защиты. Только 3% доменов правильно внедрили все три протокола. Данные результаты объясняют падение доставляемости легитимных рассылок: ужесточение требований провайдеров при массовом несоответствии отправителей создаёт системную проблему.

Alkhalil et al. [8] в публикации «Advancing Phishing Email Detection: A Comparative Study of Deep Learning Models» (Sensors, 2024) представили сравнительный анализ моделей глубокого обучения для детекции фишинговых писем: 1D-CNN, LSTM, Bi-LSTM, GRU. Исследователи зафиксировали 1 286 208 фишинговых атак во втором квартале 2023 года, что демонстрирует масштаб проблемы. Лучшие результаты показала комбинированная архитектура 1D-CNNPD + Bi-GRU. Работа демонстрирует эволюцию алгоритмов фильтрации в направлении глубокого обучения, что необходимо учитывать при адаптации легитимных рассылок.

1.1.3 Исследования в области автоматизации и имитации поведения

Направление робототехнической автоматизации процессов (RPA) и методов имитации человеческого поведения непосредственно связано с задачами настоящего исследования.

Enríquez et al. [9] в систематическом обзоре «Robotic Process Automation: A Scientific and Industrial Systematic Mapping Study» (IEEE Access, 2020)

проанализировали 54 научных публикации и 14 коммерческих RPA-инструментов по 48 функциональным категориям. Авторы определяют RPA как технологию автоматизации rule-based процессов через симуляцию взаимодействия человека с графическим интерфейсом. Выявлены пробелы в научной проработке: недостаточное развитие методов автоматического обнаружения процессов (process discovery) и отсутствие интеграции с методами искусственного интеллекта для обработки исключительных ситуаций. Работа обеспечивает методологическую основу для проектирования систем браузерной автоматизации.

Acien et al. [10] в исследовании «BeCAPTCHA-Mouse: Synthetic Mouse Trajectories and Improved Bot Detection» (Pattern Recognition, 2022) разработали нейромоторную модель для синтеза человекоподобных траекторий движения мыши. Авторы предложили два подхода: knowledge-based с использованием профилей скорости (постоянный, логарифмический, гауссов) и data-driven на основе генеративных состязательных сетей (GAN). Ключевым результатом является применение модели Sigma-Lognormal из кинематической теории человеческих движений в качестве основы для различения человека и бота. На выборке из 15 000 траекторий достигнута точность детекции 93%. Данная модель представляет теоретическую основу для реализации компонента имитации поведения пользователя.

Wang et al. [11] в работе «Using Deep Learning to Solve Google reCAPTCHA v2's Image Challenges» (IEEE IMCOM, 2020) продемонстрировали комплексный подход к автоматическому прохождению CAPTCHA: комбинация собственной свёрточной нейронной сети с Google Cloud Vision API для распознавания изображений и алгоритма имитации движений мыши (human-mimicking mouse movements) для обхода поведенческого анализа. Исследование подтверждает эффективность сочетания методов компьютерного зрения и симуляции поведения для преодоления защитных механизмов.

Park et al. [12] в публикации «Generative Agents: Interactive Simulacra of Human Behavior» (ACM UIST '23) представили архитектуру LLM-агентов, демонстрирующих сложное контекстно-зависимое поведение. Архитектура включает три компонента: memory stream (поток памяти), reflection mechanism (механизм рефлексии) и dynamic retrieval (динамическое извлечение). Агенты демонстрируют эмерджентную социальную динамику и способность адаптировать планы на основе накопленного опыта. Данный архитектурный паттерн потенциально применим для моделирования сложных сценариев взаимодействия в email-маркетинге.

1.2 Обзор отечественных исследований

Отечественные исследования в области автоматизации маркетинга, детекции ботов и проектирования распределённых систем вносят существенный вклад в теоретическую базу настоящей работы. Анализ публикаций, индексируемых в РИНЦ и журналах ВАК, позволяет учесть специфику российского научного контекста.

Яхнеева И. В. и Павлова А. В. [13] в статье «Интеллектуальная автоматизация маркетинга: угроза или возможность?» (Вопросы инновационной экономики, 2022) провели анализ потенциала технологий искусственного интеллекта для автоматизации маркетинговых процессов, включая email-рассылки и персонализацию контента. Авторы рассмотрели преимущества интеллектуальной автоматизации – масштабирование операций, прогнозирование поведения потребителей, оптимизация конверсии – и сопутствующие риски внедрения. Работа индексируется в РИНЦ и имеет более 6 цитирований, что подтверждает её востребованность в научном сообществе.

Смирнова О. С., Алымов А. С. и Баранюк В. В. [14] в публикации «Детектирование бот-программ, имитирующих поведение людей в социальной сети «ВКонтакте»» (International Journal of Open Information

Technologies, 2016) представили методы идентификации ботов, имитирующих поведение реальных пользователей. Исследователи разработали алгоритмы анализа поведенческих паттернов, позволяющие различать автоматизированную и человеческую активность. Работа выполнена при поддержке гранта РФФИ, что свидетельствует о её научной значимости. Результаты исследования релевантны для понимания принципов детекции, применяемых антибот-системами.

Емалетдинова Л. Ю. и Катасёв А. С. [15] в статье «Нейросетевая технология классификации электронных почтовых сообщений» (Вестник Казанского технологического университета, 2016) описали схему применения нейросетевой модели для бинарной классификации email-сообщений («спам» / «не спам»). Авторы использовали методологию Knowledge Discovery in Databases для построения модели, включая формирование обучающей выборки и оценку классифицирующей способности. Понимание архитектуры и принципов работы нейросетевых спам-фильтров необходимо для разработки стратегий повышения доставляемости легитимных рассылок.

Кугушева Д. С. [16] в работе «Проектирование сложного программного обеспечения с использованием микросервисной архитектуры» (Вестник науки и образования, 2020) провела сравнительный анализ микросервисного и монолитного архитектурных подходов. Автор выделяет ключевые преимущества микросервисов: гибкость развёртывания, отказоустойчивость отдельных компонентов, независимое масштабирование сервисов. Рассмотренные архитектурные паттерны применимы для проектирования backend-компонента платформы email-маркетинга, требующей обработки высоких нагрузок.

Фомин Д. С. и Бальзамов А. В. [17] в публикации «Проблематика обработки транзакций при использовании микросервисной архитектуры» (Модели, системы, сети в экономике, технике, природе и обществе, 2021) исследовали методы обеспечения согласованности данных в распределённых системах. Авторы рассмотрели механизм двухфазной фиксации (2PC) и

паттерн SAGA для управления распределёнными транзакциями. Исследование выполнено на примере системы электронной коммерции, однако результаты непосредственно применимы к проектированию надёжной системы email-рассылок, требующей гарантированной обработки задач.

Машечкин И. В., Петровский М. И. и Розинкин А. Н. [18] в статье «Система предотвращения массовых рассылок на основе алгоритмов машинного обучения» (Программирование, 2005) представили архитектуру системы автоматической фильтрации нежелательной корреспонденции. Авторы разработали комбинированный подход, объединяющий статистические алгоритмы и методы машинного обучения. Ключевым элементом является расчёт локальной вероятности лексем для персонализированной классификации сообщений. Несмотря на давность публикации, фундаментальные принципы фильтрации остаются актуальными и необходимы для понимания механизмов, препятствующих доставке легитимных рассылок.

Систематизация проанализированных источников по тематическим направлениям представлена в таблице 1.1.

Таблица 1.1 – Распределение источников по тематическим направлениям

Тематическое направление	Зарубежные	Отечественные	Всего
Browser fingerprinting и детекция	4	1	5
Email-фильтрация и аутентификация	4	2	6
Автоматизация и имитация поведения	4	1	5
Архитектура распределённых систем	–	2	2
ИТОГО	12	6	18

1.3 Общая характеристика проблемы исследования

Анализ зарубежных и отечественных исследований позволяет сформулировать комплексную характеристику проблемы снижения доставляемости email-рассылок и обосновать актуальность разработки программного комплекса с применением технологии имитации поведения пользователя.

1.3.1 Формулировка проблемы

Согласно данным отраслевых исследований, средний показатель доставляемости email-рассылок снизился с 83% в 2022 году до 76% в 2024 году. Это означает, что каждое четвёртое отправленное письмо не достигает папки «Входящие» получателя. Проблема обусловлена совокупностью факторов:

- Ужесточение политик почтовых провайдеров. С февраля 2024 года Google ввёл обязательные требования для массовых отправителей (более 5000 писем в день): наличие SPF и DKIM, настройка DMARC, показатель спам-жалоб менее 0,3%. Microsoft Outlook и Yahoo Mail внедрили аналогичные требования. При несоответствии письма отклоняются с ошибками SMTP 5xx [6, 7].
- Повышение точности систем фильтрации. Современные спам-фильтры на базе нейронных сетей и методов глубокого обучения достигают точности 99,9% [5, 8]. Фильтры анализируют сотни параметров: репутацию домена, содержание сообщения, поведение отправителя, паттерны взаимодействия получателя.
- Низкий уровень корректной настройки протоколов аутентификации. Исследование Tatang et al. [7] показало, что только 3% доменов правильно внедрили комплекс SPF + DKIM + DMARC. Около 70% политик DMARC используют режим «None», не обеспечивающий реальной защиты. Это создаёт системное противоречие: провайдеры ужесточают требования, но большинство отправителей им не соответствует.

1.3.2 Ограничения существующих подходов

Традиционные решения для email-маркетинга используют протокол SMTP для отправки сообщений. Данный подход имеет следующие ограничения:

- Зависимость от репутации домена и IP-адреса. Репутация формируется в течение длительного периода и может быть утрачена после единичного инцидента. Восстановление репутации требует значительных временных затрат.
- Сложность настройки протоколов аутентификации. Корректная настройка SPF, DKIM и DMARC требует экспертных знаний в области DNS и криптографии. Ошибки конфигурации приводят к отклонению писем.
- Уязвимость к поведенческому анализу. SMTP-отправка не предполагает взаимодействия с веб-интерфейсом, что выделяет массовые рассылки на фоне легитимной переписки между пользователями.

Альтернативный подход – браузерная автоматизация – предполагает отправку писем через веб-интерфейс почтового провайдера. Однако исследования [1–4] демонстрируют, что стандартные инструменты автоматизации легко детектируются:

- более 75% сайтов с антибот-защитой используют fingerprinting для детекции [3];
- 31,96% из Top 10K сайтов блокируют crawler'ы на основе fingerprint [2];
- стандартный Chrome без модификаций блокируется в 75% случаев поведенческим анализом [2].

1.3.3 Обоснование выбранного подхода

Проведённый анализ позволяет обосновать подход, основанный на браузерной автоматизации с применением технологии имитации поведения пользователя (Human Behavior Simulation):

- Эффективность поведенческих сигналов научно подтверждена. Amin Azad et al. [3] экспериментально установили, что добавление

движений мыши и паттернов скроллинга позволяет обходить часть систем детекции. Asien et al. [10] разработали математическую модель Sigma-Lognormal, обеспечивающую генерацию траекторий мыши, неотличимых от человеческих.

- Устранение проблемы DMARC alignment. При отправке через веб-интерфейс Gmail письма отправляются с домена @gmail.com, для которого протоколы аутентификации настроены провайдером. Это устраняет необходимость самостоятельной настройки SPF/DKIM/DMARC.
- Соответствие паттернам легитимного использования. Письма, отправленные через веб-интерфейс с имитацией поведения пользователя, неотличимы от обычной переписки с точки зрения антиспам-систем.
- Методы сокрытия автоматизации разработаны. Исследования [1, 4] определили конкретные признаки, по которым детектируется автоматизация, что позволяет разработать меры противодействия: генерация согласованных fingerprint-профилей, обход CDP-детекции, маскировка headless-режима.

1.3.4 Научная новизна и практическая значимость

Научная новизна исследования заключается в комплексном применении методов имитации поведения пользователя к задаче автоматизации email-маркетинга. Существующие исследования рассматривают отдельные аспекты: fingerprinting [1-4], спам-фильтрацию [5, 8], генерацию траекторий мыши [10, 11]. Настоящая работа интегрирует эти направления в единую архитектуру программного комплекса.

Практическая значимость состоит в разработке программного комплекса, позволяющего повысить доставляемость email-рассылок в условиях ужесточения политик почтовых провайдеров. Результаты

исследования могут быть использованы в области цифрового маркетинга, автоматизации бизнес-процессов и разработки RPA-решений.

Выводы по главе 1. Проведённый анализ 12 зарубежных и 6 отечественных научных публикаций позволил выявить актуальное состояние исследований в предметной области, сформулировать проблему и обосновать выбранный подход к её решению. Ключевые результаты анализа:

- снижение доставляемости email (с 83% до 76%) обусловлено комплексом факторов: ужесточением политик провайдеров, повышением точности фильтрации, низким уровнем корректной настройки протоколов аутентификации;
- традиционные SMTP-решения имеют системные ограничения, стандартные инструменты браузерной автоматизации легко детектируются;
- эффективность подхода Human Behavior Simulation подтверждена научными исследованиями [3, 10, 11];
- существует теоретическая и методологическая база для разработки программного комплекса, интегрирующего методы fingerprinting, имитации поведения и обхода систем детекции.

2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1 Основные методы решения проблемы и их характеристики

Для решения проблемы снижения доставляемости email-рассылок при использовании традиционных SMTP-решений в современной практике применяется комплекс методов, направленных на имитацию поведения реального пользователя и обход систем автоматической детекции. На основании анализа научных публикаций и технической документации методы систематизированы по функциональному назначению.

2.1.1 Методы генерации цифровых отпечатков браузера

Цифровой отпечаток браузера (browser fingerprint) представляет собой совокупность параметров программного и аппаратного окружения, позволяющую идентифицировать пользователя без использования cookies [1]. Исследования Vastel et al. [2] демонстрируют, что антифрод-системы почтовых провайдеров активно используют fingerprinting для выявления автоматизированной активности и связывания множества аккаунтов.

Canvas fingerprinting основан на использовании HTML5 Canvas API для генерации уникального отпечатка. Метод включает создание скрытого canvas-элемента, рендеринг текста с различными шрифтами и извлечение данных через `getImageData()` или `toDataURL()`. Различия в результатах рендеринга обусловлены особенностями GPU, графических драйверов, операционной системы и установленных шрифтов. По данным исследования FPFlow [3], 6,7% из 10 000 наиболее посещаемых сайтов используют данную технику.

WebGL fingerprinting обеспечивает более глубокую идентификацию через параметры трёхмерной графики: UNMASKED_VENDOR_WEBGL

(производитель GPU), UNMASKED_RENDERER_WEBGL (модель видеокарты), а также результаты рендеринга 3D-сцен. Исследование Cao et al. [4] продемонстрировало возможность идентификации 99,24% устройств с использованием 31 задачи WebGL-рендеринга.

Method noise injection предполагает внедрение случайного шума в результаты Canvas и WebGL-рендеринга через перехват соответствующих API. Однако данный подход имеет существенное ограничение: антибот-системы детектируют инъекцию шума путём сравнения результатов последовательных рендеров и проверки согласованности параметров (заявленный GPU должен соответствовать характеристикам рендеринга) [2].

Метод согласованной генерации профилей является наиболее эффективным подходом. Он предполагает формирование комплексного профиля браузера, в котором все параметры (User-Agent, разрешение экрана, navigator.platform, список плагинов, параметры WebGL) согласованы между собой и соответствуют реальной конфигурации устройства. Несоответствие между заявленными и фактическими характеристиками является надёжным признаком автоматизации [5].

2.1.2 Методы имитации поведения пользователя

Поведенческий анализ является одним из ключевых методов детекции автоматизации. Исследование Amin Azad et al. [5] продемонстрировало, что добавление поведенческих сигналов (движения мыши, паттерны скроллинга) позволяет обходить значительную часть систем детекции. Для имитации поведения человека применяются следующие методы.

Модель Sigma-Lognormal для генерации траекторий мыши основана на кинематической теории человеческих движений. Исследование Asien et al. [6] описывает математическую модель, учитывающую нейромоторные характеристики: профили скорости (постоянный, логарифмический, гауссов), ускорение и замедление в начале и конце движения, микродвижения и

естественное «дрожание» курсора. Модель позволяет генерировать траектории, неотличимые от человеческих с точностью детекции 93% на выборке из 15 000 траекторий.

Метод генерации траекторий по кривым Безье представляет упрощённую альтернативу. Траектория движения курсора аппроксимируется кривой Безье 3-го или 4-го порядка с добавлением случайных отклонений. Метод менее точен, но значительно проще в реализации и достаточен для большинства систем детекции.

Имитация динамики набора текста (keystroke dynamics) учитывает характерные для человека вариации интервалов между нажатиями клавиш. Параметры включают: время удержания клавиши (dwell time), интервал между нажатиями (flight time), вариативность скорости печати. Реалистичная имитация предполагает моделирование случайных пауз, ускорений при наборе знакомых слов и замедлений при сложных комбинациях символов.

Имитация паттернов скроллинга включает моделирование естественного поведения при просмотре страницы: переменная скорость прокрутки, инерция (momentum scrolling), паузы при чтении контента, плавное замедление в конце движения.

2.1.3 Методы обхода систем детекции автоматизации

Современные антибот-системы используют многоуровневый подход к детекции автоматизации [5, 7]. Для противодействия применяются специализированные методы обхода каждого уровня защиты.

Обход детекции Chrome DevTools Protocol (CDP) является критически важным, поскольку CDP-детекция считается наиболее надёжным методом выявления автоматизации [7]. Стандартные фреймворки (Playwright, Puppeteer) используют команду Runtime.Enable, которая детектируется через специальные JavaScript-конструкции. Метод обхода предполагает

модификацию фреймворка для отключения данной команды или использование альтернативных механизмов взаимодействия.

Соккрытие флага navigator.webdriver необходимо, поскольку спецификация W3C WebDriver требует установки `navigator.webdriver = true` при автоматизированном управлении браузером. Методы обхода включают: использование флага командной строки `Chrome --disable-blink-features=AutomationControlled`, JavaScript-инъекцию для переопределения свойства, модификацию бинарного файла драйвера.

Маскировка headless-режима направлена на устранение характерных признаков безголового браузера: строка «HeadlessChrome» в User-Agent, пустой список `navigator.plugins`, отсутствие определённых JavaScript API (`notification permissions`), нетипичные размеры `viewport`. Метод предполагает комплексную модификацию всех детектируемых параметров.

Устранение артефактов автоматизации включает удаление характерных переменных из глобального объекта `window` (`$cdc_`, `$wdc_` для `ChromeDriver`), изменение идентификаторов изолированных контекстов исполнения (`__puppeteer_utility_world__`), замену `sourceURL` в инжектируемых скриптах.

2.1.4 Методы балансировки нагрузки и управления аккаунтами

Ужесточение политик почтовых провайдеров (требование `spam rate < 0,3%`) делает невозможной массовую отправку с одного аккаунта [8, 9]. Необходимо распределение нагрузки между множеством аккаунтов с учётом их индивидуальных характеристик.

Метод Token Bucket rate limiting обеспечивает контроль частоты операций для каждого аккаунта. Алгоритм поддерживает «корзину» токенов, пополняемую с фиксированной скоростью; каждая операция потребляет один токен. При пустой корзине операция откладывается. Метод позволяет соблюдать суточные лимиты отправки и предотвращать всплески активности.

Метод взвешенной балансировки учитывает комплексный показатель «здоровья» аккаунта: текущая фаза прогрева (warmup), остаток суточного лимита, время с последней активности, история ошибок и блокировок. Аккаунты с более высоким рейтингом получают приоритет при распределении задач.

Метод Circuit Breaking предполагает временное исключение аккаунта из пула при превышении порога ошибок. Паттерн «размыкателя цепи» предотвращает каскадные сбои и позволяет аккаунту восстановиться перед возобновлением работы.

Сравнительный анализ рассмотренных методов по критериям эффективности, сложности реализации и устойчивости к детекции представлен в таблице 1.2.

Таблица 1.2 – Сравнительный анализ методов решения проблемы

Метод	Эффективность	Сложность	Устойчивость
Согласованная генерация fingerprint	Высокая	Высокая	Высокая
Noise injection	Средняя	Низкая	Низкая
Sigma-Lognormal траектории	Высокая	Высокая	Высокая
Кривые Безье	Средняя	Низкая	Средняя
CDP detection bypass	Высокая	Средняя	Средняя
Token Bucket rate limiting	Высокая	Низкая	Высокая
Взвешенная балансировка	Высокая	Средняя	Высокая

2.2 Основные технологии решения проблемы и их характеристики

На технологическом уровне реализация рассмотренных методов опирается на специализированные программные платформы, фреймворки и инструменты. Выбор технологий определяется требованиями к производительности, возможностям масштабирования и устойчивости к детекции.

2.2.1 Фреймворки браузерной автоматизации

Систематический обзор Enríquez et al. [10] выделяет браузерную автоматизацию как ключевую технологию RPA (Robotic Process Automation), позволяющую имитировать взаимодействие человека с веб-интерфейсами. Для задач автоматизации email-маркетинга рассматриваются три основных фреймворка.

Playwright (Microsoft, 2020) представляет собой современный фреймворк, ставший де-факто стандартом для сложной браузерной автоматизации. Поддерживает браузеры Chromium, Firefox, WebKit через единый API. Ключевым преимуществом является встроенная поддержка изолированных browser contexts, позволяющих управлять множеством независимых сессий в рамках одного процесса браузера. Потребление оперативной памяти составляет 50–200 МБ на сессию [7].

Puppeteer (Google, 2017) обеспечивает управление браузером Chromium через Chrome DevTools Protocol. Фреймворк ограничен языком JavaScript и не имеет встроенной поддержки изоляции контекстов. Потребление памяти выше – 100–300 МБ на сессию. Преимуществом является более низкоуровневый доступ к CDP, что может быть полезно для специфических задач.

Selenium является наиболее зрелым решением с поддержкой всех популярных браузеров и языков программирования. Однако фреймворк не предоставляет механизмов изоляции сессий и характеризуется наибольшим потреблением памяти (200–500 МБ на сессию). Архитектура WebDriver легко детектируется антибот-системами.

Сравнительный анализ фреймворков представлен в таблице 2.1.

Таблица 2.1 – Сравнительный анализ фреймворков браузерной автоматизации

Характеристика	Playwright	Puppeteer	Selenium
Браузеры	Chromium, Firefox, WebKit	Chromium	Все основные
Языки	JS, Python, Java, C#	JavaScript	Все популярные
Изоляция контекстов	Встроенная	Ограниченная	Отсутствует
RAM на сессию	50–200 МБ	100–300 МБ	200–500 МБ
Auto-waiting	Лучший	Частичный	Базовый

2.2.2 Технологии сокрытия автоматизации

Стандартные фреймворки автоматизации легко детектируются антибот-системами, что требует применения специализированных технологий сокрытия [5, 7].

rebrowser-playwright представляет собой модифицированную версию Playwright с критическими патчами для обхода детекции. Ключевые модификации включают: исправление утечки Runtime.Enable (детектируемой Cloudflare, DataDome), замену sourceURL-идентификаторов в инжектируемых скриптах, изменение имени изолированного контекста исполнения. Библиотека является drop-in replacement для стандартного Playwright и активно поддерживается (последнее обновление – май 2025).

puppeteer-extra-plugin-stealth включает 17 модулей обхода детекции: navigator.webdriver, chrome.runtime, navigator.plugins, WebGL vendor и другие. Существенным недостатком является отсутствие обновлений с 2022 года, что позволило антибот-системам изучить и нейтрализовать применяемые техники.

undetected-chromedriver для Python/Selenium модифицирует бинарный файл ChromeDriver, удаляя характерные переменные (\$cdc_, \$wdc_). Ограничением является работоспособность только при программной навигации; взаимодействие через графический интерфейс приводит к детекции.

2.2.3 Брокеры сообщений для распределённой обработки задач

Архитектура браузерной автоматизации предполагает асинхронную обработку задач множеством воркеров. Выбор брокера сообщений определяется требованиями к латентности, пропускной способности и гарантиям доставки [11].

Redis Streams обеспечивает минимальную латентность (<1 мс), критичную для интерактивных браузерных сессий. Механизм consumer groups позволяет распределять задачи между воркерами с гарантией at-least-once

delivery. Операционная сложность минимальна благодаря использованию Redis в качестве единой инфраструктурной зависимости (кэш, очереди, блокировки).

RabbitMQ представляет классическое решение для очередей сообщений с широкими возможностями маршрутизации. Латентность составляет 5–50 мс, что приемлемо для большинства сценариев. Дисковая персистенция обеспечивает сохранность сообщений при сбоях.

Apache Kafka оптимизирован для обработки миллионов сообщений в секунду с гарантией exactly-once delivery. Высокая операционная сложность и латентность (10–100 мс) делают решение избыточным для задач браузерной автоматизации среднего масштаба.

Сравнительные характеристики брокеров представлены в таблице 2.2.

Таблица 2.2 – Сравнительный анализ брокеров сообщений

Критерий	Redis Streams	RabbitMQ	Apache Kafka
Латентность	<1 мс	5–50 мс	10–100 мс
Пропускная способность	~1M ops/s	~50K msg/s	Millions/s
Операционная сложность	Низкая	Средняя	Высокая
Гарантии доставки	At-least-once	Configurable	Exactly-once
Персистенция	In-memory + AOF	Disk-based	Distributed log

2.2.4 Системы оркестрации процессов

Управление долгоживущими браузерными сессиями требует механизмов отслеживания состояния, обработки сбоев и восстановления [12].

Temporal.io обеспечивает durable execution – автоматическое сохранение состояния на каждом шаге workflow и восстановление при сбоях. Каждая браузерная операция может быть оформлена как отдельная Activity с настраиваемой политикой повторных попыток. Механизм heartbeat позволяет отслеживать долгоживущие сессии. Решение используется в production-системах Netflix, Stripe, Coinbase.

Celery представляет собой распределённую систему выполнения задач для Python с поддержкой множества брокеров (Redis, RabbitMQ). Простота интеграции с Django делает решение привлекательным для Python-based

проектов. Ограничением является отсутствие встроенного механизма сохранения состояния workflow.

Apache Airflow предназначен для оркестрации пакетных задач по расписанию. Отсутствие поддержки real-time задач и долгоживущих процессов делает решение неприменимым для браузерной автоматизации.

Сравнение систем оркестрации представлено в таблице 2.3.

Таблица 2.3 – Сравнительный анализ систем оркестрации

Характеристика	Temporal.io	Celery	Airflow
Durable execution	Да	Нет	Нет
Real-time задачи	Да	Да	Нет
Долгоживущие процессы	Да	Ограничено	Нет
Восстановление состояния	Автоматическое	Ручное	Ручное
Интеграция с Python/Django	SDK	Нативная	SDK

2.3 Алгоритм выбора методов и технологий решения проблемы

На основании проведённого анализа методов и технологий разработан алгоритм выбора оптимального технологического стека для системы автоматизации email-маркетинга. Алгоритм учитывает специфику предметной области, требования к масштабируемости и ограничения ресурсов.

2.3.1 Критерии выбора

При выборе методов и технологий учитываются следующие критерии:

- 1) Устойчивость к детекции – способность обходить современные антибот-системы почтовых провайдеров. Критерий является приоритетным, поскольку детекция приводит к блокировке аккаунтов.
- 2) Потребление ресурсов – объём оперативной памяти и процессорного времени на один браузерный процесс. Определяет количество одновременных сессий на сервере.

- 3) Латентность операций – время отклика при выполнении браузерных операций и обмене сообщениями между компонентами.
- 4) Операционная сложность – затраты на развёртывание, настройку и сопровождение инфраструктуры.
- 5) Масштабируемость – возможность горизонтального расширения системы при росте нагрузки.
- 6) Зрелость и поддержка – наличие документации, сообщества, регулярных обновлений.

2.3.2 Пошаговый алгоритм выбора

Алгоритм выбора технологического стека включает следующие этапы:

Этап 1. Анализ требований к нагрузке. Определяется целевое количество одновременных браузерных сессий и суточный объём операций. Для систем с нагрузкой до 50 одновременных сессий рекомендуется упрощённая архитектура (Celery + Redis). При нагрузке 50–500 сессий – полноценный оркестратор (Temporal.io). Свыше 500 сессий – распределённая архитектура с Kubernetes.

Этап 2. Выбор фреймворка браузерной автоматизации. При требовании изоляции сессий и минимального потребления памяти выбирается Playwright. При необходимости низкоуровневого доступа к CDP – Puppeteer. Selenium рассматривается только при наличии legacy-кода или требований к специфическим браузерам.

Этап 3. Выбор технологии сокрытия автоматизации. При выборе Playwright рекомендуется rebrowser-playwright как наиболее актуальное решение с активной поддержкой. Для Puppeteer – puppeteer-extra-plugin-stealth с учётом ограничений. Для Selenium – undetected-chromedriver.

Этап 4. Выбор брокера сообщений. При требовании минимальной латентности (<5 мс) и нагрузке до миллиона операций в день – Redis Streams. При требовании гарантированной доставки и сложной маршрутизации –

RabbitMQ. При обработке миллионов операций с требованием exactly-once delivery – Apache Kafka.

Этап 5. Выбор системы оркестрации. При необходимости durable execution и автоматического восстановления – Temporal.io. Для простых сценариев с Python – Celery. Airflow не рекомендуется для задач браузерной автоматизации.

Этап 6. Валидация выбора. Проверяется совместимость выбранных компонентов, соответствие нефункциональным требованиям, наличие необходимых интеграций.

2.3.3 Обоснование выбранного технологического стека

Применение разработанного алгоритма для предлагаемого программного комплекса с учётом требований (до 24 одновременных сессий, минимальная латентность, Python-based разработка) позволяет обосновать следующий выбор технологий:

Фреймворк браузерной автоматизации: Playwright. Обоснование: минимальное потребление памяти (50–200 МБ), встроенная изоляция контекстов, поддержка Python, лучший в классе механизм auto-waiting.

Технология сокрытия: rebrowser-playwright. Обоснование: актуальные патчи для обхода CDP-детекции, drop-in replacement для стандартного Playwright, активная поддержка.

Брокер сообщений: Redis Streams. Обоснование: минимальная латентность (<1 мс), механизм consumer groups, низкая операционная сложность, использование Redis для множества задач (кэш, блокировки, очереди).

Система оркестрации: кастомный оркестратор на базе Celery и Asyncio. Обоснование: интеграция с Django, возможность реализации специфической логики управления браузерными процессами (1 аккаунт = 1 процесс), персистентность сессий.

Итоговая архитектура технологического стека представлена в таблице 2.4.

Таблица 2.4 – Итоговый технологический стек

Компонент	Технология	Обоснование выбора
Браузерная автоматизация	Playwright	Изоляция контекстов, низкий RAM
Соккрытие автоматизации	rebrowser-playwright	Актуальные патчи CDP, активная поддержка
Брокер сообщений	Redis Streams	Минимальная латентность, consumer groups
Оркестрация	Celery + Asyncio	Интеграция с Django, гибкость
Веб-фреймворк	Django 5.x	Зрелая экосистема, ORM, ASGI
База данных	PostgreSQL	Надёжность, JSON-поддержка

Таким образом, разработанный алгоритм выбора позволяет систематически определить оптимальный набор методов и технологий с учётом специфики задачи и ресурсных ограничений. Выбранный технологический стек обеспечивает баланс между устойчивостью к детекции, производительностью и операционной сложностью.

3 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ

3.1 Общая характеристика предлагаемого решения

На основании проведённого анализа существующих методов и технологий в автоматизации email-маркетинга предлагается разработка программного комплекса, предназначенного для автоматизированной отправки электронных писем с применением технологии имитации поведения пользователя (Human Behavior Simulation). Ключевой особенностью предлагаемого решения является использование браузерной автоматизации вместо традиционных SMTP-протоколов, что позволяет преодолеть ограничения, связанные с ужесточением политик почтовых провайдеров [1, 2].

Архитектурно программный комплекс построен по принципу *гибридного модульного монолита* (Modular Monolith) с элементами микросервисной архитектуры и событийно-ориентированным взаимодействием компонентов. Выбор данного архитектурного паттерна обусловлен необходимостью обеспечения баланса между целостностью данных, характерной для монолитных систем, и гибкостью масштабирования, присущей микросервисам [3, 4].

Система декомпозирована на следующие функциональные слои:

Ядро системы (Core Layer) представляет собой монолитное приложение, реализованное на базе веб-фреймворка Django 5.x. Данный компонент отвечает за бизнес-логику управления учётными записями, хранение метаданных рассылок и предоставление программного интерфейса (API) для взаимодействия с внешними модулями. Выбор Django обусловлен его зрелой экосистемой, встроенной ORM для работы с реляционными базами данных и поддержкой асинхронных интерфейсов через Django Channels (ASGI).

Исполнительный слой (Execution Layer) реализован в виде специализированного оркестратора, управляющего пулом изолированных

процессов-воркеров. Каждый воркер представляет собой отдельный процесс операционной системы, инкапсулирующий экземпляр headless-браузера Playwright. Архитектурное решение «один аккаунт – один браузерный процесс» продиктовано требованиями к изоляции сессий и необходимостью жёсткого контроля над потреблением оперативной памяти [5]. В отличие от стандартных решений на базе Celery, данная архитектура обеспечивает персистентность браузерных сессий между операциями.

Слой вспомогательных сервисов включает модуль сбора контактных данных (Lead Generation), вынесенный в отдельный микросервис на базе FastAPI. Данный компонент работает с собственной базой данных и очередями задач, что позволяет масштабировать его независимо от основного приложения и обеспечивает отказоустойчивость при сбоях.

Взаимодействие компонентов системы осуществляется посредством следующих механизмов:

- Redis Streams – для организации очередей задач отправки с поддержкой consumer groups и гарантией at-least-once delivery;
- Redis Pub/Sub – для доставки уведомлений о событиях в реальном времени через WebSocket-соединения;
- PostgreSQL – в качестве основного хранилища бизнес-данных с разделением на схемы для различных подсистем.

Выбор Redis Streams в качестве брокера сообщений обусловлен требованиями к минимальной латентности (<1 мс), критичной для интерактивных браузерных сессий, при сохранении возможности горизонтального масштабирования через механизм consumer groups [6]. Использование Playwright в качестве фреймворка браузерной автоматизации продиктовано его преимуществами перед альтернативами: поддержка изолированных browser contexts, низкое потребление памяти (50–200 МБ на сессию) и наличие патчей для сокрытия факта автоматизации [7, 8].

Общая схема взаимодействия компонентов представлена на рисунке 1. Пользовательские запросы поступают через API-слой, преобразуются в задачи и помещаются в очередь Redis Streams. Оркестратор извлекает задачи из очереди и распределяет их между доступными воркерами с учётом текущей нагрузки и состояния аккаунтов. Каждый воркер выполняет операции в изолированном браузерном контексте, имитируя поведение реального пользователя.

Предлагаемое архитектурное решение позволяет преодолеть ограничения существующих систем за счёт следующих особенностей:

- использование веб-интерфейса почтового провайдера вместо SMTP-протокола исключает проблемы с аутентификацией SPF/DKIM/DMARC;
- генерация уникальных цифровых отпечатков браузера для каждого аккаунта обеспечивает устойчивость к детекции [9];
- имитация паттернов поведения человека (траектории движения мыши, динамика набора текста) снижает вероятность классификации как автоматизированной активности [10].

3.2 Функциональные требования к предлагаемому решению

Функциональные требования определяют перечень операций и возможностей, которые должна обеспечивать система. На основе анализа предметной области и выявленных ограничений существующих решений сформулированы требования, сгруппированные по функциональным модулям. Каждое требование идентифицировано уникальным кодом (FR – Functional Requirement) и содержит описание ожидаемого поведения системы.

3.2.1 Модуль управления аккаунтами и безопасностью

Данный модуль отвечает за жизненный цикл учётных записей почтовых сервисов, генерацию цифровых отпечатков и прохождение проверок безопасности. Требования к модулю представлены в таблице 3.1.

Таблица 3.1 – Функциональные требования к модулю управления аккаунтами

Код	Описание требования
FR-001	Система должна обеспечивать хранение и управление учётными данными почтовых аккаунтов, включая статус жизненного цикла (ожидание авторизации, активен, заблокирован, требует повторной авторизации) и данные токенов доступа
FR-002	Система должна генерировать уникальные цифровые отпечатки браузера для каждого аккаунта, включая User-Agent, разрешение экрана, параметры WebGL, Canvas fingerprint и характеристики аппаратного обеспечения
FR-003	Система должна выполнять автоматизированный вход в почтовый сервис через браузер, обрабатывая сценарии: ввод пароля, подтверждение резервной почты, двухфакторная аутентификация, выбор аккаунта
FR-004	Система должна автоматически проходить проверки безопасности, включая решение CAPTCHA (через интеграцию с внешними сервисами) и верификацию по SMS

Требование FR-002 основано на исследованиях в области browser fingerprinting [9, 11], демонстрирующих возможность идентификации пользователей по совокупности параметров браузера. Генерация согласованных профилей, включающих User-Agent, разрешение экрана, параметры WebGL и Canvas, позволяет имитировать работу с различных физических устройств.

Реализация требования FR-003 базируется на модели конечного автомата (Finite State Machine), обрабатывающего различные сценарии авторизации: стандартный вход с паролем, подтверждение через резервную почту, двухфакторная аутентификация и восстановление доступа. Данный подход обеспечивает устойчивость к изменениям интерфейса почтового провайдера.

3.2.2 Модуль рассылок и оркестрации

Модуль рассылок обеспечивает создание кампаний, распределение нагрузки между аккаунтами и физическую отправку сообщений. Ключевой

особенностью является интеллектуальная балансировка, учитывающая состояние каждого аккаунта. Требования представлены в таблице 3.2.

Таблица 3.2 – Функциональные требования к модулю рассылок

Код	Описание требования
FR-005	Система должна обеспечивать создание кампаний рассылки на основе загружаемых файлов формата CSV с поддержкой Spintax-шаблонов для уникализации текста сообщений
FR-006	Система должна распределять задачи на отправку между аккаунтами на основе алгоритма интеллектуальной балансировки, учитывающего суточные лимиты, время простоя и фазу прогрева аккаунта
FR-007	Система должна обеспечивать отправку писем через веб-интерфейс почтового сервиса с имитацией поведения человека: траектории движения мыши, паузы при вводе текста, естественные задержки между действиями
FR-008	Система должна поддерживать приоритетную очередь задач с разделением на быстрые ответы (высокий приоритет) и массовые рассылки (стандартный приоритет)

Требование FR-006 реализует алгоритм интеллектуальной балансировки нагрузки, ранжирующий аккаунты по комплексному показателю «здоровья». Алгоритм учитывает: текущую фазу прогрева (warmup), остаток суточного лимита отправки, время с последней активности и историю ошибок. Такой подход минимизирует риск блокировки аккаунтов при сохранении высокой пропускной способности системы.

Имитация поведения человека (FR-007) основана на модели Sigma-Lognormal [10], описывающей кинематику движений мыши. Реализация включает: генерацию траекторий по кривым Безье с переменным ускорением, микродвижения и «дрожание» курсора, паузы перед кликом. Динамика набора текста моделируется с учётом характерных для человека вариаций интервалов между нажатиями клавиш.

3.2.3 Модуль коммуникации

Модуль коммуникации обеспечивает двустороннее взаимодействие: синхронизацию входящей корреспонденции и управление диалогами с получателями. Требования к модулю представлены в таблице 3.3.

Таблица 3.3 – Функциональные требования к модулю коммуникации

Код	Описание требования
FR-009	Система должна синхронизировать историю входящих сообщений через API почтового провайдера и сохранять их в локальном хранилище с возможностью полнотекстового поиска
FR-010	Система должна предоставлять интерфейс для просмотра цепочек писем (тредов) и отправки ответов в контексте существующего диалога
FR-011	Система должна обеспечивать автоматическое или ручное назначение входящих тредов на операторов для последующей обработки
FR-012	Система должна доставлять уведомления о событиях (изменение статуса задачи, новое входящее сообщение) в реальном времени через WebSocket-соединения

Требование FR-012 реализуется через протокол WebSocket с использованием Django Channels. Асинхронная доставка уведомлений обеспечивает мгновенное отображение изменений статуса задач и новых входящих сообщений без необходимости периодического опроса сервера (polling), что снижает нагрузку на инфраструктуру.

3.3 Нефункциональные требования к предлагаемому решению

Нефункциональные требования определяют качественные характеристики системы: производительность, масштабируемость, безопасность и отказоустойчивость. Данные требования критичны для обеспечения стабильной работы системы в условиях высокой нагрузки и противодействия системам детекции автоматизации.

3.3.1 Требования к производительности и масштабируемости

Требования данной группы определяют пропускную способность системы и возможности её расширения. Учитывая ресурсоёмкость браузерной автоматизации (потребление 50–200 МБ оперативной памяти на каждый экземпляр браузера [7]), особое внимание уделено оптимизации использования ресурсов.

Таблица 3.4 – Нефункциональные требования к производительности

Код	Описание требования
NFR-001	Система должна поддерживать одновременную работу до 24 экземпляров браузеров на одном узле оркестрации
NFR-002	Система должна ограничивать размер дискового кэша каждого браузера (не более 50 МБ) для предотвращения переполнения дискового пространства
NFR-003	Архитектура системы должна поддерживать горизонтальное масштабирование через контейнеризацию компонентов с возможностью независимого масштабирования слоёв

Ограничение в 24 одновременных браузерных процесса (NFR-001) определено эмпирически на основе тестирования на серверах с 16 ГБ оперативной памяти. При среднем потреблении 500 МБ на процесс суммарное использование памяти составляет около 12 ГБ, оставляя резерв для операционной системы и вспомогательных сервисов.

3.3.2 Требования к безопасности и анонимности

Требования безопасности направлены на защиту от детекции автоматизации со стороны антифрод-систем почтовых провайдеров. Исследования [8, 9, 12] демонстрируют, что современные системы детекции используют многоуровневый анализ: проверку navigator.webdriver, fingerprint consistency, поведенческий анализ.

Таблица 3.5 – Нефункциональные требования к безопасности

Код	Описание требования
NFR-004	Система должна скрывать признаки автоматизации браузера: удаление флага navigator.webdriver, подмена параметров WebGL/Canvas, модификация идентификаторов CDP
NFR-005	Всё взаимодействие с внешними сервисами должно осуществляться через прокси-серверы, индивидуально привязанные к каждому аккаунту
NFR-006	Браузерные сессии (Cookies, Local Storage, IndexedDB) должны сохраняться в персистентном хранилище для исключения повторных авторизаций

Реализация требования NFR-004 основана на использовании библиотеки rebrowser-playwright [8], включающей патчи для обхода детекции Chrome DevTools Protocol (CDP). Ключевые модификации включают: отключение команды Runtime.Enable, замену идентификаторов utility world, подмену sourceURL в инжектируемых скриптах.

Требование NFR-005 обеспечивает привязку уникального IP-адреса к каждому аккаунту через индивидуальные прокси-серверы. Это предотвращает корреляцию аккаунтов по сетевым характеристикам и соответствует рекомендациям по обеспечению анонимности браузерных сессий [12].

3.3.3 Требования к отказоустойчивости и надёжности

Требования отказоустойчивости обеспечивают стабильную работу системы при возникновении сбоев и предотвращают деградацию производительности при длительной эксплуатации.

Таблица 3.6 – Нефункциональные требования к отказоустойчивости

Код	Описание требования
NFR-007	Система должна автоматически перезапускать задачи, находящиеся в статусе обработки более 15 минут, с возвратом в очередь для повторного выполнения
NFR-008	При обнаружении блокировки аккаунта или недоступности прокси система должна автоматически обновлять статус аккаунта и очищать связанную очередь задач
NFR-009	Система должна отслеживать и завершать «зомби-процессы» браузеров, оставшиеся после аварийного завершения воркеров

Механизм автоматического перезапуска задач (NFR-007) реализован через периодическую проверку состояния очереди. Задачи, находящиеся в статусе обработки более 15 минут, считаются «зависшими» и возвращаются в очередь для повторного выполнения. Данный подход обеспечивает гарантию eventually-consistent delivery. При этом система ведёт учёт количества попыток выполнения каждой задачи: после трёх неудачных попыток задача перемещается в отдельную очередь для ручного анализа, что предотвращает бесконечные циклы перезапуска и позволяет оперативно выявлять системные проблемы.

Требование NFR-009 направлено на предотвращение утечек памяти, характерных для долгоживущих браузерных процессов. Оркестратор отслеживает состояние дочерних процессов и принудительно завершает «зомби-процессы», оставшиеся после аварийного завершения воркеров. Мониторинг осуществляется с периодичностью 60 секунд посредством

анализа таблицы процессов операционной системы. Дополнительно реализован механизм превентивного перезапуска браузерных контекстов после обработки заданного количества задач (по умолчанию — 100 операций), что позволяет минимизировать накопление фрагментированной памяти и поддерживать стабильную производительность системы в режиме длительной эксплуатации.

Сводная информация о сформулированных требованиях представлена в таблице 3.7.

Таблица 3.7 – Сводная таблица требований к программному комплексу

Категория	Количество	Идентификаторы
Управление аккаунтами	4	FR-001 – FR-004
Рассылки и оркестрация	4	FR-005 – FR-008
Коммуникация	4	FR-009 – FR-012
Производительность	3	NFR-001 – NFR-003
Безопасность	3	NFR-004 – NFR-006
Отказоустойчивость	3	NFR-007 – NFR-009
ИТОГО	21	12 FR + 9 NFR

Таким образом, сформулированный комплекс из 12 функциональных и 9 нефункциональных требований задаёт чёткие рамки для последующей проектной и разработческой деятельности. Требования учитывают специфику предметной области, выявленные в ходе анализа ограничения существующих решений и результаты научных исследований в области browser fingerprinting и имитации поведения пользователя.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы проведён комплексный анализ проблемы снижения доставляемости email-рассылок и сформированы требования к программному комплексу автоматизации email-маркетинга с применением технологии имитации поведения пользователя.

По результатам первой главы выполнен анализ 12 зарубежных и 6 отечественных научных публикаций. Выявлено актуальное состояние исследований в области браузерной идентификации, систем фильтрации электронной почты и технологий имитации поведения. Сформулирована проблема: снижение доставляемости с 83% до 76% обусловлено ужесточением политик провайдеров, повышением точности фильтрации и низким уровнем корректной настройки протоколов аутентификации. Обосновано применение подхода Human Behavior Simulation на основе экспериментальных данных научных исследований.

По результатам второй главы проведён сравнительный анализ методов и технологий решения проблемы. Систематизированы методы генерации цифровых отпечатков (согласованная генерация профилей, noise injection), методы имитации поведения (модель Sigma-Lognormal, кривые Безье, keystroke dynamics), методы обхода детекции (CDP bypass, сокрытие webdriver). Выполнен сравнительный анализ технологий: фреймворков браузерной автоматизации (Playwright, Puppeteer, Selenium), брокеров сообщений (Redis Streams, RabbitMQ, Kafka), систем оркестрации (Temporal.io, Celery, Airflow). Разработан шестиэтапный алгоритм выбора технологического стека.

По результатам третьей главы сформулированы требования к программному комплексу. Определена архитектура гибридного модульного монолита с элементами микросервисов и событийно-ориентированным взаимодействием. Сформулированы 12 функциональных требований (FR-001 – FR-012), охватывающих модули управления аккаунтами, рассылок и

коммуникации. Определены 9 нефункциональных требований (NFR-001 – NFR-009), обеспечивающих производительность, безопасность и отказоустойчивость системы.

Основные результаты исследования:

- 1) установлено, что эффективность подхода Human Behavior Simulation научно подтверждена: добавление поведенческих сигналов позволяет обходить значительную часть систем детекции;
- 2) обоснован выбор технологического стека: Playwright с rebrowser-patches для браузерной автоматизации, Redis Streams для организации очередей, гибридная архитектура для обеспечения масштабируемости;
- 3) сформулирован полный комплекс требований (21 требование), достаточный для перехода к этапу проектирования и разработки.

Перспективы дальнейшей работы связаны с реализацией программного комплекса в соответствии с сформулированными требованиями в рамках выпускной квалификационной работы. Планируется разработка прототипа системы, проведение экспериментальной оценки показателей доставляемости, оптимизация алгоритмов имитации поведения.

Таким образом, поставленная цель исследования достигнута, все задачи выполнены. Результаты работы создают теоретический фундамент для практической реализации программного комплекса автоматизации email-маркетинга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Vastel, A. FP-Scanner: The Privacy Implications of Browser Fingerprint Inconsistencies / A. Vastel, P. Laperdrix, W. Rudametkin, R. Rouvoy // Proceedings of the 27th USENIX Security Symposium. – USENIX Association, 2018. – P. 135–150.
2. Vastel, A. FP-Crawlers: Studying the Resilience of Browser Fingerprinting to Block Crawlers / A. Vastel, W. Rudametkin, R. Rouvoy, X. Blanc // Proceedings of the 2020 NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb '20). – Internet Society, 2020. – DOI: 10.14722/madweb.2020.23010.
3. Amin Azad, B. Web Runner 2049: Evaluating Third-Party Anti-bot Services / B. Amin Azad, O. Starov, P. Laperdrix, N. Nikiforakis // Detection of Intrusions and Malware, and Vulnerability Assessment. DIMVA 2020. Lecture Notes in Computer Science. – Vol. 12223. – Springer, 2020. – P. 135–159. – DOI: 10.1007/978-3-030-52683-2_7.
4. Amin Azad, B. Taming the Shape Shifter: Detecting Anti-fingerprinting Browsers / B. Amin Azad, O. Starov, P. Laperdrix, N. Nikiforakis // DIMVA 2020. Lecture Notes in Computer Science. – Vol. 12223. – Springer, 2020. – P. 160–170. – DOI: 10.1007/978-3-030-52683-2_8.
5. Dada, E. G. Machine learning for email spam filtering: review, approaches and open research problems / E. G. Dada, J. S. Bassi, H. Chiroma [et al.] // Heliyon. – 2019. – Vol. 5, № 6. – Art. e01802. – DOI: 10.1016/j.heliyon.2019.e01802.
6. Nightingale, J. S. Email Authentication Mechanisms: DMARC, SPF and DKIM : NIST Technical Note 1945 / J. S. Nightingale. – National Institute of Standards and Technology, 2017. – DOI: 10.6028/NIST.TN.1945.
7. Tatang, D. The Evolution of DNS-based Email Authentication: Measuring Adoption and Finding Flaws / D. Tatang, F. Zettl, T. Holz // Proceedings of the

24th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '21). – ACM, 2021. – P. 354–369. – DOI: 10.1145/3471621.3471842.

8. Alkhalil, Z. Advancing Phishing Email Detection: A Comparative Study of Deep Learning Models / Z. Alkhalil, C. Hewage, L. Nawaf, I. Khan // Sensors. – 2024. – Vol. 24, № 7. – Art. 2077. – DOI: 10.3390/s24072077.

9. Enríquez, J. G. Robotic Process Automation: A Scientific and Industrial Systematic Mapping Study / J. G. Enríquez, A. Jiménez-Ramírez, F. J. Domínguez-Mayo, J. A. García-García // IEEE Access. – 2020. – Vol. 8. – P. 39113–39129. – DOI: 10.1109/ACCESS.2020.2974934.

10. Acien, A. BeCAPTCHA-Mouse: Synthetic Mouse Trajectories and Improved Bot Detection / A. Acien, A. Morales, J. Fierrez, R. Vera-Rodriguez // Pattern Recognition. – 2022. – Vol. 127. – Art. 108643. – DOI: 10.1016/j.patcog.2022.108643.

11. Wang, D. Using Deep Learning to Solve Google reCAPTCHA v2's Image Challenges / D. Wang, M. Moh, T.-S. Moh // 2020 14th International Conference on Ubiquitous Information Management and Communication (IMCOM). – IEEE, 2020. – P. 1–5. – DOI: 10.1109/IMCOM48794.2020.9001774.

12. Park, J. S. Generative Agents: Interactive Simulacra of Human Behavior / J. S. Park, J. C. O'Brien, C. J. Cai [et al.] // Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23). – ACM, 2023. – DOI: 10.1145/3586183.3606763.

13. Яхнеева, И. В. Интеллектуальная автоматизация маркетинга: угроза или возможность? / И. В. Яхнеева, А. В. Павлова // Вопросы инновационной экономики. – 2022. – Т. 12, № 1. – С. 155–166. – DOI: 10.18334/vinec.12.1.114116.

14. Смирнова, О. С. Детектирование бот-программ, имитирующих поведение людей в социальной сети «ВКонтакте» / О. С. Смирнова, А. С. Алымов, В. В. Баранюк // International Journal of Open Information Technologies. – 2016. – Т. 4, № 8. – С. 55–60.

15. Емалетдинова, Л. Ю. Нейросетевая технология классификации электронных почтовых сообщений / Л. Ю. Емалетдинова, А. С. Катасёв // Вестник Казанского технологического университета. – 2016. – Т. 19, № 13. – С. 148–152.
16. Кугушева, Д. С. Проектирование сложного программного обеспечения с использованием микросервисной архитектуры / Д. С. Кугушева // Вестник науки и образования. – 2020. – № 6-2 (84). – С. 14–18.
17. Фомин, Д. С. Проблематика обработки транзакций при использовании микросервисной архитектуры / Д. С. Фомин, А. В. Бальзамов // Модели, системы, сети в экономике, технике, природе и обществе. – 2021. – № 2 (38). – С. 89–98.
18. Машечкин, И. В. Система предотвращения массовых рассылок на основе алгоритмов машинного обучения / И. В. Машечкин, М. И. Петровский, А. Н. Розинкин // Программирование. – 2005. – № 5. – С. 34–51.
19. Google Email Sender Guidelines [Электронный ресурс]. – URL: <https://support.google.com/a/answer/81126> (дата обращения: 15.12.2024).
20. Playwright Documentation [Электронный ресурс]. – URL: <https://playwright.dev/docs/intro> (дата обращения: 15.12.2024).
21. rebrowser-patches: Playwright patches for browser automation stealth [Электронный ресурс]. – URL: <https://github.com/rebrowser/rebrowser-patches> (дата обращения: 15.12.2024).
22. Redis Streams Documentation [Электронный ресурс]. – URL: <https://redis.io/docs/data-types/streams/> (дата обращения: 15.12.2024).
23. Temporal.io Documentation [Электронный ресурс]. – URL: <https://docs.temporal.io/> (дата обращения: 15.12.2024).
24. Django Documentation [Электронный ресурс]. – URL: <https://docs.djangoproject.com/> (дата обращения: 15.12.2024).